

MINISTRY OF EDUCATION AND RESEARCH
OF THE REPUBLIC OF MOLDOVA
Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software Engineering and Automation

Vasilean Vitalie
Group FAF-241

Report

Laboratory Work No. 10
Operating Systems
Topic: Add Your Own System Call: trace (xv6)

Checked by:
univ. assist. / lect. Patricia Reițman
Department of Software Engineering and Automation
FCIM, UTM

Chisinau – 2026

Contents

1 INTRODUCTION AND GOAL **3**

1.1 Introduction 3

1.2 Lab Objective 3

2 PART 1: SYSTEM CALL ARCHITECTURE AND USERLAND DISPATCHER **4**

2.1 Userland System Call Configuration 4

2.2 Implementation of `user/trace.c` 4

2.3 Makefile Integration 5

3 PART 2: KERNEL IMPLEMENTATION OF TRACE **6**

3.1 Process Control Block Extension (`kernel/proc.h`) 6

3.2 Kernel Syscall Handler (`kernel/sysproc.c`) 6

3.3 Syscall Interception and Dispatching (`kernel/syscall.c`) 6

3.4 Inheritance Across Process Creation (`kernel/proc.c`) 7

4 PART 3: VERIFICATION AND OBSERVATIONAL ANALYSIS **9**

4.1 Experimental Testing in QEMU 9

4.2 Analysis of Verification Run (`trace 32 grep hello README`) 9

4.3 Answers to Required Observation Questions 9

5 CONCLUSIONS **11**

REFERENCES **12**

1 INTRODUCTION AND GOAL

1.1 Introduction

Inside the xv6 operating system running on the 64-bit RISC-V architecture, the boundary between user applications and the kernel is guarded strictly by hardware privilege modes. User programs execute in unprivileged User Mode (U-mode), whereas kernel code executes in privileged Supervisor Mode (S-mode). When a user program requires an operating system service, it cannot jump directly to a kernel memory address; rather, it loads the requested service number into a dedicated CPU register (`a7`) and executes the RISC-V `ecall` (environment call) instruction.

Implementing a custom system call provides an end-to-end understanding of how hardware trap vectors, trapframe register saving, system call table lookup, argument unmarshaling, and process control block management cooperate to service user requests safely.

1.2 Lab Objective

The primary objective of this laboratory work is to design, implement, and verify a new system call within the xv6 kernel: `SYS_trace` (system call number 23). The specific goals comprise:

- configuring the userland interface by updating `kernel/syscall.h`, `user/user.h`, and the automated assembly stub generator `user/usys.pl`;
- developing the user-space diagnostic tool `user/trace.c` and registering it in the build system (`Makefile`);
- extending the process control block (`struct proc`) to store a per-process bitmask (`trace_mask`);
- implementing the kernel service routine `sys_trace()` in `kernel/sysproc.c` and wiring it into the dispatch jump table in `kernel/syscall.c`;
- intercepting system call returns and logging filtered trace messages formatted with the process PID, system call name, and return value;
- ensuring that process tracing configurations inherit across the process hierarchy through `fork()`;
- analyzing bitwise mask configurations and evaluating interactive execution traces inside QEMU.

2 PART 1: SYSTEM CALL ARCHITECTURE AND USERLAND DISPATCHER

2.1 Userland System Call Configuration

To establish the system call interface, definitions were added across three distinct header and generator files:

a) **Numeric Syscall Identifier (`kernel/syscall.h`):** A new identifier constant `SYS_trace` was allocated with numeric value `23`:

```
#define SYS_trace 23
```

b) **C Function Prototype (`user/user.h`):** The prototype was declared in the user-space system call interface:

```
int trace(int);
```

c) **Assembly Stub Generator (`user/usys.pl`):** An entry was appended to the Perl generation script:

```
entry("trace");
```

During compilation, this emits the RISC-V machine assembly stub in `user/usys.S`:

```
.global trace
trace:
 li a7, SYS_trace
 ecall
 ret
```

Figure 2.1: Figure 2.1 – Userland system call interface configuration

2.2 Implementation of `user/trace.c`

The user program `user/trace.c` accepts a bitmask argument, activates tracing in the kernel for the calling process via `trace()`, and immediately transfers execution to the target command using `exec()`:

```
// user/trace.c
#include "kernel/types.h"
#include "kernel/stat.h"
#include "user/user.h"

int
main(int argc, char *argv[])
{
  if (argc < 3) {
    fprintf(2, "usage: trace mask command [args...]\n");
    exit(1);
  }

  if (trace(atoi(argv[1])) < 0) {
    fprintf(2, "trace: failed\n");
    exit(1);
```

```
  }

  exec(argv[2], argv + 2);
  fprintf(2, "trace: exec %s failed\n", argv[2]);
  exit(1);
}
```

Figure 2.2: Figure 2.2 – Source code of user/trace.c

2.3 Makefile Integration

The user utility was registered under the `UPROGS` rule in the xv6 `Makefile` to include `_trace` in the generated disk image (`fs.img`):

```
UPROGS=\
    $U/_cat\
    $U/_echo\
    $U/_forktest\
    $U/_grep\
    $U/_init\
    $U/_kill\
    $U/_ln\
    $U/_ls\
    $U/_mkdir\
    $U/_rm\
    $U/_sh\
    $U/_stressfs\
    $U/_usertests\
    $U/_grind\
    $U/_wc\
    $U/_zombie\
    $U/_trace\
```

Figure 2.3: Figure 2.3 – Registration of user/trace in the xv6 Makefile

3 PART 2: KERNEL IMPLEMENTATION OF TRACE

3.1 Process Control Block Extension (kernel/proc.h)

To track which system calls should be logged for any specific process, an integer field `trace_mask` was added to `struct proc`:

```
// kernel/proc.h
struct proc {
  struct spinlock lock;
  enum procstate state;
  void *chan;
  int killed;
  int xstate;
  int pid;
  struct proc *parent;
  uint64 kstack;
  uint64 sz;
  pagetable_t pagetable;
  struct trapframe *trapframe;
  struct context context;
  struct file *ofile[NOFILE];
  struct inode *cwd;
  char name[16];
  int trace_mask; // Bitmask of system calls to trace
};
```

Figure 3.1: Figure 3.1 – Field addition in kernel/proc.h

3.2 Kernel Syscall Handler (kernel/sysproc.c)

The kernel service function `sys_trace()` retrieves the integer bitmask passed by the user from register `a0` using `argint()` and saves it into the current process descriptor:

```
// kernel/sysproc.c
uint64
sys_trace(void)
{
  int mask;
  argint(0, &mask);
  myproc()->trace_mask = mask;
  return 0;
}
```

Figure 3.2: Figure 3.2 – Implementation of sys_trace in kernel/sysproc.c

3.3 Syscall Interception and Dispatching (kernel/syscall.c)

The dispatch table `syscalls[]` was updated to link `SYS_trace` to `sys_trace`. Additionally, a string lookup table `syscallnames[]` was created to convert numeric system call identifiers into human-readable names.

Inside the central dispatch routine `syscall()`, after invoking the system call handler and saving its return value into `trapframe->a0`, the kernel tests whether the corresponding bit in `trace_mask` is set:

```
// kernel/syscall.c
static char *syscallnames[] = {
  [SYS_fork]    "fork",
  [SYS_exit]    "exit",
  [SYS_wait]    "wait",
  [SYS_pipe]    "pipe",
  [SYS_read]    "read",
  [SYS_kill]    "kill",
  [SYS_exec]    "exec",
  [SYS_fstat]   "fstat",
  [SYS_chdir]   "chdir",
  [SYS_dup]     "dup",
  [SYS_getpid]  "getpid",
  [SYS_sbrk]    "sbrk",
  [SYS_pause]   "pause",
  [SYS_uptime]  "uptime",
  [SYS_open]    "open",
  [SYS_write]   "write",
  [SYS_mknod]   "mknod",
  [SYS_unlink]  "unlink",
  [SYS_link]    "link",
  [SYS_mkdir]   "mkdir",
  [SYS_close]   "close",
  [SYS_sync]    "sync",
  [SYS_trace]   "trace",
};

void
syscall(void)
{
  int num;
  struct proc *p = myproc();

  num = p->trapframe->a7;
  if(num > 0 && num < NELEM(syscalls) && syscalls[num]) {
    p->trapframe->a0 = syscalls[num]();
    if ((p->trace_mask >> num) & 1) {
      printk("%d: syscall %s -> %ld\n",
             p->pid, syscallnames[num], p->trapframe->a0);
    }
  } else {
    printk("%d %s: unknown sys call %d\n", p->pid, p->name, num);
    p->trapframe->a0 = -1;
  }
}
```

Figure 3.3: Figure 3.3 – Interceptor and table bindings in kernel/syscall.c

3.4 Inheritance Across Process Creation (`kernel/proc.c`)

To ensure that child processes created via `fork()` continue to be traced, the `trace_mask` is copied from the parent process (`p`) to the newly allocated child process descriptor (`np`) inside `kfork()` in `kernel/proc.c`:

```
// kernel/proc.c (inside kfork)
safestrcpy(np->name, p->name, sizeof(p->name));
np->trace_mask = p->trace_mask; // Inherit parent's trace mask
```

```
4 pid = np->pid;
```

Figure 3.4: Figure 3.4 – Inheritance of trace mask in kernel/proc.c

4 PART 3: VERIFICATION AND OBSERVATIONAL ANALYSIS

4.1 Experimental Testing in QEMU

The modified kernel was compiled and booted under QEMU to execute the diagnostic test suites:

```
$ trace 32 grep hello README
3: syscall read -> 1023
3: syscall read -> 965
3: syscall read -> 453
3: syscall read -> 0

$ trace 2147483647 grep hello README
3: syscall trace -> 0
3: syscall exec -> 0
3: syscall open -> 3
3: syscall read -> 1023
3: syscall read -> 965
3: syscall read -> 453
3: syscall read -> 0
3: syscall close -> 0

$ trace 2 usertests forkforkfork
...
usertests: all tests passed
```

Figure 4.1: Figure 4.1 – Interactive verification of trace system call

4.2 Analysis of Verification Run (`trace 32 grep hello README`)

During the execution of `trace 32 grep hello README`, the `read` system call is invoked four times, returning buffer lengths `1023`, `965`, `453`, and `0`.

$$\sum \text{Bytes Read} = 1023 + 965 + 453 = 2441 \text{ bytes} \tag{4.1}$$

This matches the exact byte size of `README` reported by `ls` in the xv6 root filesystem (`README 2 2 2441`). The final return value of `0` represents standard POSIX End of File (EOF).

4.3 Answers to Required Observation Questions

a) **Why does `trace 32 grep hello README` display only `read` calls?** The mask value 32 corresponds to 2^5 ($1 \ll 5$), which sets bit 5 in the bitmask. In `kernel/syscall.h`, `SYS_read` is assigned index 5. The bitwise expression `(p->trace_mask >> num) & 1` evaluates to true strictly when `num == 5`. Other system calls executed by `grep` (such as `open` with index 15 and `close` with index 21) have their respective bits set to 0 and are filtered out.

b) **Which line of code enables tracing for distinct child PIDs in `forkforkfork`?** The line added to the `fork()` implementation in `kernel/proc.c`:

```
    np->trace_mask = p->trace_mask;
```

This propagates the parent's tracing mask to the child's process control block. When child processes subsequently execute system calls, their calls are intercepted and printed with their respective newly assigned PIDs.

c) **Why does the trace output interleave between "hi" and the newline in `trace 65536 echo hi`?** The integer 65536 equals 2^{16} ($1 \ll 16$), which activates tracing for `SYS_write` (index 16). The `echo` utility in `user/echo.c` writes its output using two separate calls:

- First call: `write(1, "hi", 2)` outputs the argument text. As soon as the call finishes, the kernel logs:

```
1        hi66: syscall write -> 2
```

- Second call: `write(1, "\n", 1)` transmits the newline character, causing the second trace line to begin on a new terminal row:

```
1        66: syscall write -> 1
```

d) **Mask Calculation for Intercepting Exclusively `open` (15) and `close` (21):** To monitor exclusively `open` and `close`, bits 15 and 21 must be set:

$$\text{Mask} = 2^{15} + 2^{21} = 32768 + 2097152 = \mathbf{2129920} \tag{4.2}$$

Verification command:

```
1    $ trace 2129920 grep hello README
2    3: syscall open -> 3
3    3: syscall close -> 0
```

5 CONCLUSIONS

In this laboratory work, a custom system call (`SYS_trace`) was engineered and integrated into the xv6-riscv operating system.

The implementation spanned all levels of the system call hierarchy: userland stub generation in `usys.pl`, trap instruction execution via `ecall`, parameter unmarshaling from trapframe registers via `argint()`, kernel process descriptor state management in `struct proc`, and trap return logging in `kernel/syscall.c`. Furthermore, configuring process inheritance in `kernel/proc.c` ensured that tracing persists seamlessly across process fork trees.

This work solidified theoretical principles of hardware privilege isolation, trap dispatching, and kernel state propagation in a functional multi-core operating system.

REFERENCES

[1] Russ Cox, Frans Kaashoek, and Robert Morris, *xv6: a simple, Unix-like teaching operating system*, MIT CSAIL, 2024.

[2] Andrew S. Tanenbaum and Herbert Bos, *Modern Operating Systems*, Pearson, 4th Edition, 2014.

[3] David A. Patterson and John L. Hennessy, *Computer Organization and Design RISC-V Edition: The Hardware Software Interface*, Morgan Kaufmann, 2nd Edition, 2020.

[4] Department of Software Engineering and Automation, *Operating Systems Lab 10: Add Your Own System Call: trace*, FCIM, Technical University of Moldova, 2026.